

Lesson 8 Worksheet: Driving the Path – ANSWER KEY

Part A: Driving Function Design

1. Two pieces of state threaded through the driving functions:

State	Type	Purpose
position	tuple	The robot's current (row, col.) on the grid
heading	int	The direction the robot faces: 0=N, 1=E, 2=S, 3=W

2. Three driving functions:

Function Name	Input	Output
desired_heading	position, next_pos (tuples)	Returns the heading number (0-3) to reach the next position
turn_to	heading, desired (ints)	Returns the new heading after turning right until it matches desired
drive_path	path, position, heading	Returns the final (position, heading) after driving the whole path

3. Why do `turn_to()` and `drive_path()` need to `return` their updated values?

Because `position` and `heading` are local variables inside each function – changes made to a parameter inside a function do not affect anything outside it. The only way the caller can see the updated values is if the function returns them and the caller reassigns its own variables from that return value.

4. What functions does `drive_path()` call to physically move the robot?

`turn_right()`, `track_until_cross()`, and `clear_intersection()` – the driving toolkit built in Module 2 and reused in Module 3.

Part B: `desired_heading()` Exercises

<i>Position</i>	<i>Next Position</i>	<i>row_diff</i>	<i>col_diff</i>	<i>Desired Heading</i>
(0, 0)	(1, 0)	$1 - 0 = 1$	$0 - 0 = 0$	2 (S)
(2, 1)	(1, 1)	$1 - 2 = -1$	$1 - 1 = 0$	0 (N)
(1, 2)	(1, 3)	0	1	1 (E)
(3, 3)	(3, 2)	0	-1	3 (W)
(0, 1)	(1, 1)	1	0	2 (S)
(2, 0)	(2, 1)	0	1	1 (E)
(3, 2)	(2, 2)	-1	0	0 (N)
(1, 3)	(1, 2)	0	-1	3 (W)

Part C: Trace `turn_to()` by Hand

Each pass: turn right, add 1, wrap 4 $\rightarrow$ 0. Stop when `heading == desired`, then return it.

<i>Current</i>	<i>Desired</i>	<i>heading values during turn_to</i>	<i>Total Turns</i>	<i>Returned</i>
0 (N)	1 (E)	0 $\rightarrow$ 1	1	1
0 (N)	2 (S)	0 $\rightarrow$ 1 $\rightarrow$ 2	2	2
1 (E)	1 (E)	already there	0	1
1 (E)	0 (N)	1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 0 (wrap)	3	0
2 (S)	3 (W)	2 $\rightarrow$ 3	1	3
2 (S)	0 (N)	2 $\rightarrow$ 3 $\rightarrow$ 0 (wrap)	2	0
3 (W)	2 (S)	3 $\rightarrow$ 0 (wrap) $\rightarrow$ 1 $\rightarrow$ 2	3	2
3 (W)	1 (E)	3 $\rightarrow$ 0 (wrap) $\rightarrow$ 1	2	1
0 (N)	3 (W)	0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3	3	3
1 (E)	3 (W)	1 $\rightarrow$ 2 $\rightarrow$ 3	2	3

- How many ran the while loop zero times? 1 (E $\rightarrow$ E)
- Which scenarios triggered the wrap from 4 back to 0? The four rows where the loop crosses the 3 $\rightarrow$ 0 boundary (1 $\rightarrow$ 0, 2 $\rightarrow$ 0, 3 $\rightarrow$ 2, 3 $\rightarrow$ 1)
- What would go wrong without `return heading`? `turn_to` computes the correct value internally but throws it away when the function ends. The caller's `heading` variable never updates, so every subsequent turn calculation starts from the wrong heading.

Part D: drive_path() Tracing

First path: [(1, 0), (2, 0), (2, 1), (2, 2)]

Starting position: (0, 0), Starting heading: 0 (N)

Step	next_pos	Desired	Current	turn_to heading values	New Heading	New Position
1	(1, 0)	2 (S)	0 (N)	0 → 1 → 2	2 (S)	(1, 0)
2	(2, 0)	2 (S)	2 (S)	already there	2 (S)	(2, 0)
3	(2, 1)	1 (E)	2 (S)	2 → 3 → 0 (wrap) → 1	1 (E)	(2, 1)
4	(2, 2)	1 (E)	1 (E)	already there	1 (E)	(2, 2)

drive_path() returns ((2, 2), 1).

Note on steps 2 and 4: Already facing the right way, so drive_path calls clear_intersection() before track_until_cross().

Second path: [(3, 2), (3, 1), (2, 1), (1, 1), (0, 1)]

Starting position: (3, 3), Starting heading: 1 (E)

Step	next_pos	Desired	Current	turn_to heading values	New Heading	New Position
1	(3, 2)	3 (W)	1 (E)	1 → 2 → 3	3 (W)	(3, 2)
2	(3, 1)	3 (W)	3 (W)	already there	3 (W)	(3, 1)
3	(2, 1)	0 (N)	3 (W)	3 → 0 (wrap)	0 (N)	(2, 1)
4	(1, 1)	0 (N)	0 (N)	already there	0 (N)	(1, 1)
5	(0, 1)	0 (N)	0 (N)	already there	0 (N)	(0, 1)

drive_path() returns ((0, 1), 0).

Part E: Code Completion

```
HEADING_NAMES = ["N", "E", "S", "W"]
```

```
def desired_heading(position, next_pos):  
    row_diff = next_pos[0] - position[0]  
    col_diff = next_pos[1] - position[1]  
    if row_diff == -1:  
        return 0  
    elif col_diff == 1:  
        return 1  
    elif row_diff == 1:  
        return 2  
    elif col_diff == -1:  
        return 3
```

```
def turn_to(heading, desired):  
    while heading != desired:  
        turn_right()  
        heading = heading + 1  
        if heading == 4:  
            heading = 0  
    return heading
```

```
def drive_path(path, position, heading):  
    for next_pos in path:  
        needed = desired_heading(position, next_pos)  
        if heading == needed:  
            clear_intersection()  
        heading = turn_to(heading, needed)  
        track_until_cross()  
        position = next_pos  
    return position, heading
```

Blanks (in order): 0, 0, 1, 1, 0, 1, 2, 3 (desired_heading's indices and return values) – desired, turn_right, 1, 4, 0, heading (turn_to) – path, position, next_pos, needed, clear_intersection, heading, needed, next_pos, position, heading (drive_path)

Why does `drive_path()` call `clear_intersection()` only when the robot is already facing the right way?

When no turn is needed, the robot is still sitting on the previous intersection's crossing line.

`clear_intersection()` drives 8 cm forward to clear the cross so `track_until_cross()` can look for the NEXT crossing line without immediately re-detecting the current one. When a turn does happen, the turning motion (inside `turn_right()`) already clears the cross, so no extra call is needed.

Why does the loop iterate directly with `for next_pos in path:` instead of skipping elements?

Because the path does not include the starting position – every element is a new cell the robot needs to drive to. The caller supplies the current position separately as a parameter, so the path only contains the destinations to visit.

Reflection

`drive_path()` reuses your Module 2/3 driving toolkit instead of controlling motors directly. Why is this a good design choice?

Reusing the toolkit means `drive_path()` doesn't need to reimplement line tracking, crossing detection, or turning from scratch. `turn_right()` and `track_until_cross()` already handle following a line and making precise turns. If you improve the toolkit (like better PID tuning), every function that calls it benefits automatically. It also keeps each layer focused on one job: the sensor/driving toolkit handles low-level sensor-based driving, and the Lesson 8 functions handle high-level path navigation.

Part F (Optional Extension): The Navigator Class

1. Fill in the blanks:

```

class Navigator:

    def __init__(self, start, heading):
        self.position = start
        self.heading = heading
        self.line_track = LineTrack()

    def desired_heading(self, next_pos):
        row_diff = next_pos[0] - self.position[0]
        col_diff = next_pos[1] - self.position[1]
        # ... same four if/elif branches

    def turn_to(self, desired):
        while self.heading != desired:
            self.line_track.turn_right()
            self.heading = self.heading + 1
            if self.heading == 4:
                self.heading = 0

        # Note: no return statement needed here. Why not?
        # Because self.heading IS the object's own state -- updating it
        # directly changes what the object remembers, with no need to
        # hand a copy back to the caller.

    def drive_path(self, path):
        for next_pos in path:
            needed = self.desired_heading(next_pos)
            if self.heading == needed:
                self.line_track.clear_intersection()
            self.turn_to(needed)
            self.line_track.track_until_cross()
            self.position = next_pos

```

2. Equivalent class-version call:

```

navigator.drive_path(path)

```

(no reassignment needed – `navigator.position` and `navigator.heading` are already updated)

3. Why doesn't the class version's `drive_path` need a `return` statement?

Because it modifies `self.position` and `self.heading` directly – those attributes belong to the `navigator` object itself, so the object already reflects the new state the moment the method finishes. The functions version has no object to update, so it must hand the new values back explicitly through `return`.

